

CSCE 614 COMPUTER ARCHITECTURE · MODULE 05 · LECTURE NOTES

Caches: The Memory Wall

The gap that shapes every modern machine.

MODULE OUTCOMES

- Explain the memory wall and its historical origin.
- Explain temporal and spatial locality and why caches work at all.
- Analyse direct-mapped, set-associative, and fully associative caches.
- Classify misses as compulsory, capacity, or conflict.
- Measure your own machine's cache hierarchy.

1 The memory wall

Between roughly 1980 and 2005, processor performance improved at about 50% per year while DRAM latency improved at about 7% per year. Two exponentials with different exponents diverge, and over twenty-five years this one diverged by more than two orders of magnitude.

The result is the defining constraint of modern architecture: a processor capable of several operations per cycle, attached to a memory that takes two to three hundred cycles to answer. Note that DRAM *bandwidth* has improved much faster than latency — you can obtain a great deal of data per second, but the first byte is still slow to arrive. That asymmetry is why prefetching (Module 06) is so valuable: it converts a latency problem into a bandwidth problem.

Level	Typical size	Latency	Notes
Registers	~1 KB	0 cycles	Named directly by the ISA.
L1 data cache	32–48 KB	~4 cycles	Per core, split from the L1 instruction cache.
L2 cache	0.5–2 MB	~12 cycles	Usually per core.
L3 cache	8–64 MB	~40 cycles	Shared between cores.
DRAM	8–128 GB	~250 cycles	Shared, and physically far.
SSD	0.5–4 TB	~10 ⁵ cycles	A different regime.

Each level is roughly ten times larger and three to five times slower than the one above it. That geometric structure is a deliberate design: it gives an average access time close to the fastest level while providing capacity close to the largest.

2 Why caching works at all

A cache is a bet that future accesses resemble past ones. The bet pays because real programs exhibit two forms of locality.

- **Temporal locality.** An item used recently is likely to be used again soon. Loop counters, hot function code, frequently traversed structures.
- **Spatial locality.** Items near a recently used item are likely to be used soon. Array traversal, struct fields accessed together, sequential instruction fetch.

Caches exploit temporal locality by *retaining* data, and spatial locality by fetching an entire **cache line** — typically 64 bytes — rather than a single word. The line is the unit of transfer, the unit of coherence (Module 11), and the unit in which you should think about data layout.

THE CONSEQUENCE FOR DATA LAYOUT

Touching one byte brings in 64. If you use all 64, the fetch cost is amortised over 64 bytes of useful work. If you use 4 and skip the rest, you have wasted 94% of the bandwidth you paid for.

This single observation justifies struct-of-arrays layouts, hot/cold field splitting, and most of Module 06.

3 Cache organisation

3.1 Address breakdown

```

63                12 11                6 5                0
+-----+-----+-----+
|          TAG          |   INDEX   |  OFFSET  |
+-----+-----+-----+

OFFSET : byte within the line      (64B line -> 6 bits)
INDEX  : which set to search        (64 sets  -> 6 bits)
TAG    : compared with stored tags to confirm a hit

```

For a 32 KB, 8-way set-associative cache with 64-byte lines: $32768/64 = 512$ lines, divided into $512/8 = 64$ sets, requiring 6 index bits.

WHY THE INDEX COMES FROM THE MIDDLE

If the index were taken from the high-order bits, an entire contiguous array would map to a single set and the cache would be useless for exactly the access pattern it most needs to serve.

Taking it from the middle means consecutive lines map to consecutive sets, so sequential data spreads evenly across the whole cache.

The price is that addresses separated by $(\text{number of sets} \times \text{line size})$ collide — which is the mechanism behind conflict misses, and the reason power-of-two strides are dangerous.

3.2 Associativity

Policy	Placement	Lookup cost	Trade-off
Direct-mapped	Exactly one possible location	One tag comparison	Fastest and simplest; badly vulnerable to conflict misses.
N-way set associative	Any of N ways within one set	N comparisons in parallel	The standard compromise. 8-way is typical for L1, 16-way for L3.
Fully associative	Anywhere in the cache	Compare every tag	No conflict misses at all, but the comparison hardware limits this to very small caches — TLBs and victim caches.

4 The three Cs

Miss type	Cause	What helps
Compulsory	The first reference to a line — it could not possibly have been cached.	Prefetching (Module 06); larger lines if spatial locality is good.
Capacity	The working set is larger than the cache, so lines are evicted before reuse.	Blocking or tiling to shrink the active working set; reduce the data itself.
Conflict	Too many active lines map to the same set, so they evict each other despite free capacity elsewhere.	Padding to change the stride; higher associativity; reorganising the access pattern.

Classifying a miss before trying to fix it saves a great deal of wasted effort, because the three have entirely different remedies. Conflict misses in particular are counterintuitive: the cache has ample free space and is still missing on every access.

4.1 The power-of-two stride trap

Consider traversing a column of a 1024×1024 array of 4-byte floats. Successive elements are 4096 bytes apart.

With 64-byte lines and 64 sets, the set index repeats every $64 \times 64 = 4096$ bytes. Every element of the column therefore maps to the *same set*. An 8-way cache can hold eight of them; the ninth evicts the first, and by the time the traversal wraps around, nothing useful remains.

THE CACHE IS 98% EMPTY AND COMPLETELY INEFFECTIVE

You are using 8 lines out of 512 and missing on every single access, while the rest of the cache sits idle.

The fix is padding. Allocate rows of 1032 floats instead of 1024 and use only the first 1024. The stride is no longer a power of two, successive column elements map to different sets, and the cache works as intended.

The cost is about 0.8% extra memory for what can be a several-fold speedup. Any time a 2D array has a power-of-two row length and column access is slow, suspect this first.

5 Measuring your own hierarchy

You can determine every cache size on your machine without consulting documentation, which is Project 1's central experiment.

```
// Build a cyclic pointer chain through a buffer of `size` bytes,
// visiting lines in RANDOM order, then time a long traversal.
for (size = 4*KB; size <= 256*MB; size *= 2) {
    build_random_chain(buf, size);
    printf("%8zu KB  %6.1f cycles/access\n",
           size/1024, time_chase(buf, ITERS));
}
```

The output shows plateaus separated by sharp steps. Each plateau is a cache level, and each step marks the point where the working set exceeded that level's capacity:

Working set	Cycles per access	Interpretation
32 KB	~4	Fits in L1.
64 KB–512 KB	~12	Spilled to L2.
2 MB–16 MB	~40	Spilled to L3.
32 MB+	~200	Spilled to DRAM.

RANDOMISE THE CHAIN, OR YOU WILL MEASURE NOTHING

A sequential pointer chase is perfectly predicted by the hardware prefetcher, which fetches lines before they are requested. The latency is hidden entirely and the curve is flat at every size — you learn nothing about capacity.

Randomising the order defeats the prefetcher, exposes the true latency of each level, and makes the steps appear. The fact that the sequential version is flat is itself the most convincing demonstration of what prefetching does, which is Module 06.

6 Replacement and write policy

- **Replacement.** LRU is close to optimal in practice but expensive to implement exactly above 2-way associativity. Real caches use pseudo-LRU approximations, tree-based schemes, or even random replacement at high associativity, where the difference is small.
- **Write-through** propagates every write to the next level immediately. Simple and keeps levels consistent, at the cost of substantial write bandwidth.
- **Write-back** writes only to the cache, marks the line dirty, and writes it out when evicted. Far less traffic, and the standard choice.
- **Write-allocate** fetches a line on a write miss before modifying it.

WRITE-ALLOCATE HAS A COST PEOPLE MISS

Writing a large array you will never read still consumes *read* bandwidth, because every write miss first fetches the line it is about to overwrite completely.

For streaming writes — clearing a buffer, writing output you will not revisit — **non-temporal stores** bypass the cache and skip the fetch, roughly halving memory traffic. They also avoid evicting useful data, which is often the larger benefit.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Ulrich Drepper — What Every Programmer Should Know About Memory	Sections 2–3 cover this module in far greater depth. The single most valuable free document on the subject.
Onur Mutlu — Memory Hierarchy and Caches lectures	Organisation, the three Cs, and replacement policies.
MIT 6.004 — The Memory Hierarchy	A clean introduction to associativity and address breakdown.
Igor Ostrovsky — Gallery of Processor Cache Effects	Nine short experiments, each isolating one cache behaviour. Run all of them — it takes an afternoon and is worth a week of reading.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement the random pointer chase and plot cycles per access against working-set size on log axes. Identify L1, L2, L3, and DRAM, and compare with your processor's specification.
- 2 Repeat with a *sequential* chain. Explain why the curve is nearly flat, and what that tells you about prefetching.
- 3 Determine your cache line size: access an array with strides of 1, 2, 4, ..., 256 bytes and find the stride beyond which time per access stops increasing.
- 4 Determine associativity: access N addresses all mapping to the same set and find the N at which the time per access jumps.
- 5 Reproduce the power-of-two stride trap: traverse the columns of a 1024×1024 float matrix, then pad the row length to 1032 and repeat. Report the speedup and the cache-miss counts for both.

- 6 Compare writing a 1 GB buffer with ordinary stores and with non-temporal stores. Measure total memory traffic for each and explain the difference.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is the memory wall, and what two divergent rates produced it?
- 2 Define temporal and spatial locality, and say which cache mechanism exploits each.
- 3 Why is the set index taken from the middle bits of an address rather than the top?
- 4 Name the three Cs and give a distinct fix for each.
- 5 Explain precisely why a 1024-float row length makes column traversal catastrophically slow, and why 1032 fixes it.
- 6 Why must a pointer chase be randomised to measure cache capacity?
- 7 Why does writing an array you never read still consume read bandwidth?